

Window-Chained Longest Common Subsequence: Common Event Matching in Sequences

Chunyao Song, Tingjian Ge

Department of Computer Science, University of Massachusetts, Lowell, MA, USA

{csong, ge}@cs.uml.edu

Abstract—Sequence data is prevalent, and event processing over sequences is increasingly important in this big data era, drawing much attention from both research and industry. In this paper, we address a novel problem, which is to find common event subsequences from two long sequences. This problem is well motivated, with applications in diverse domains. We propose the window-chained longest common subsequence (WCLCS) semantics, and argue that the traditional longest common subsequence (LCS) cannot serve this need. We then devise efficient algorithms to solve this problem by reducing it to a graph problem. We also propose two more methods to improve the performance: one is based on informed search and exploration, and the other is an approximation algorithm with accuracy guarantees. We finally carry out a systematic experimental evaluation using two real-world datasets and some synthetic datasets.

I. INTRODUCTION

Sequence data is everywhere. Sequences can be over space or over time. In particular, any devices, sensors, smartphones, or software programs that continuously or intermittently emit readings or event logs over time may essentially produce very large sequence data. In general, a sequence can be considered as an ordered list of tuples, where each tuple bears several attributes. We often extract interesting *basic events* from the tuple sequence and form a sequence of characters, over which sophisticated algorithms, such as complex event matching and frequent pattern mining, can be efficiently run.

Example 1. *Hospitals often log patients' test measurements and activities related to their diseases for a period of time. For example, in the widely used diabetes dataset [24], the information logged for diabetes patients include insulin dose, blood glucose measurement (pre- or post- breakfast, lunch, or supper), meal ingestion amount, and exercise activity amount. The interesting basic events may include insulin types (e.g., R for regular and N for NPH insulin), blood glucose level (e.g., B for pre-breakfast blood glucose high), and meal-ingestion/exercise-activity level (e.g., M for a more-than-usual meal and E for less-than-usual exercise). Then we get a sequence of characters. Suppose that there are two patients who initially only had prediabetes (i.e., light symptoms), but whose condition worsened into very serious diabetes. By comparing their logs, doctors find the common event sequences that may help understand the progress timing of the disease and possible factors that lead to the progress.*

In Example 1, the log of each of the two patients is a long sequence, and the problem is to find a common subsequence between them. Longest Common Subsequence (LCS) is a classical Computer Science problem [4]. Namely, the LCS of two sequences s_1 and s_2 is the longest sequence that is a sub-

sequence of both s_1 and s_2 . For instance, if $s_1 = ECHEACEA$ and $s_2 = DHEDCAH$, then $LCS(s_1, s_2) = HECA$, as it can be derived from both s_1 and s_2 by deleting some characters without changing the order of the remaining characters.

However, LCS is not suitable for finding interesting common event sequences from two (long) event sequences, since two matching events in a sequence can be arbitrarily far away from each other. That is, there can be an arbitrary gap in time or number of events between two matching events. Such matching in general is not meaningful, which is why complex event processing systems require a window constraint (e.g., [6, 1, 18]). In the patient log example, suppose one patient overeats in almost every meal for a week, while the other patient only overeats in one meal every three days. LCS would produce a matching that cannot differentiate their overeating behaviors, while clearly the first patient has a greater health risk.

Example 2. *In crime/security investigation, the investigator has two suspects' location/activity logs for the past few years. The investigator suspects that the two closely worked together (in the same places, plotting the crime) for an unknown number of days in a row, although they do not live or work together otherwise. By examining their logs, we wish to efficiently find out the consecutive days when they worked together.*

We can represent the basic events (places/locations) as characters and get a sequence for each suspect. Again LCS result does not help here because it may find matches where the two suspects go to the same set of places but over different periods of time. This, unfortunately, does not indicate that they physically work together, and can be quite common especially if the two suspects live in the same area.

We propose Window-Chained Longest Common Subsequence (WCLCS) for this type of common event matching. Intuitively, we have *window* units based on application semantics and examine matching across two windows from two sequences respectively. For instance, we may have window sizes of one week in Example 1 (i.e., comparing two patients' one week's activities and tests gives a good indication of their states), and one day in Example 2 (as we know they worked together for a few days, but do not know how many).

As sequences are long, just matching one window from each sequence is inadequate. Matching windows can be *chained* and we look for the longest total matching length. This is illustrated in Fig. 1. There may be two constraints: (1) the minimum matching density d_{min} of each window, and (2) the maximum gap g_{max} between two matching windows. For instance, in Examples 1 and 2, we may have $d_{min} = 0.6$. Example 1 may have $g_{max} = \infty$ (i.e., no gap constraint), while

Example 2 may have g_{max} only a few hours, adding a little slack between contiguous days/windows.

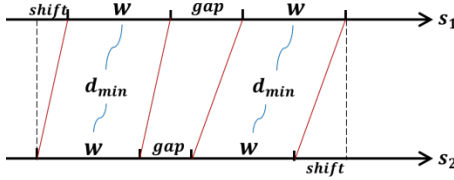


Fig. 1. Illustrating WCLCS, where a window (marked w) in sequence s_1 matches a window in s_2 with density at least d_{min} , followed by a gap at each sequence that is no bigger than g_{max} , then followed by another window match. A shift is the absolute time difference between two matching windows.

A final (optional) constraint in WCLCS is called the maximum *shift* s_{max} . Intuitively, a *shift* is the (absolute) time difference between two matching windows in two sequences respectively. This is illustrated in Fig. 1. In Example 2, s_{max} could be one day since the two suspects should work together on the same day. We list yet another two application examples in different domains below, just to show how widely relevant this problem is.

Example 3. In *Competitive Intelligence* [11], one is interested in comparing the stock prices of two major competitors in a particular market analysis. Given the hourly stock price history time series of companies A and B (or two countries), we can derive two sequences that indicate whether the price increases or not over the previous hour. With a window size of one week and $s_{max} = 1$ month, we can use WCLCS to find the positive correlations between the price changes of the two companies (or countries). They might signify common trends in the industry (for two companies) or in the global economy (for two countries). Parameter s_{max} allows us to catch when one company (or country)'s price change follows or traces the other's, with a delay of up to 1 month.

Example 4. Many car insurance companies install sensors in the vehicles of their customers [20]. These sensors record the customer's driving behavior such as speed, how sharp a stop is, and left or right turns. Given the sequences of two "bad" drivers who frequently have accidents, one may want to find out their common driving patterns through WCLCS. The window size could be 4 hours (typically a trip lasts no more than 4 hours), and the minimum density d_{min} could be 0.6.

In all these motivating examples, window constraints are necessary to preserve the semantics of local matching, while the window *chain* semantics extends it globally over the whole sequences. We focus on stored sequences and time series (rather than unbounded streams), which is the case with all the motivating applications.

Our contributions. In this paper we first formulate the WCLCS problem (Section II). Then we break it down into two sub-problems: (1) How to obtain the longest common subsequence within each two-dimensional time window (one dimension from each sequence)? We call it per-window LCS or WLCS. (2) Based on the result in (1), how to find an optimal chain of windows that satisfy the constraints?

For the first sub-problem (WLCS), we propose two algorithms. The first one is a non-trivial extension of the classical algorithm for LCS. The second algorithm is inspired from the

first one, but is much more efficient. It focuses on pairs of locations in the two sequences where the letters match (which we call *matching points*). It also resorts to (external) sorting and spatial indexing for efficiency (Section III).

To solve the second sub-problem, we perform a series of conversions on the problem: initially to a longest path problem in a graph with weighted vertices, and finally to a single-source shortest path problem in a directed acyclic graph with weighted edges, allowing efficient algorithms (Section IV).

Then we propose two novel methods to improve performance. The first one carries out informed search and exploration of the space based on upper bounds of the matching length of window chains. The idea is like "best-first" search, finding the optimal window chain while minimizing the search space exploration, which belongs to the broad category of A* search [13]. We prove that each matching window needs to be visited and expanded *at most once*.

The second performance improvement is an approximation algorithm that samples the matching points, speeding up our algorithms in multiple ways. We prove the probabilistic accuracy guarantee and the tradeoff between performance and accuracy. Our experiments show that this algorithm is very effective, improving performance by over an order of magnitude while achieving nearly perfect accuracy (Section V).

We perform a comprehensive empirical evaluation using two real-world datasets and synthetic datasets in Section VI. Finally, we survey related work in Section VII.

II. PRELIMINARIES AND PROBLEM FORMULATION

In this section, we start with some preliminaries on the classical Longest Common Subsequence (LCS) problem and its solution. Then we introduce the formulation of our problem.

A. Preliminaries on Longest Common Subsequence

A *sequence* (a.k.a. *string*) $s = s[1], s[2], \dots, s[n]$ consists of n *letters* (a.k.a. *characters* or *items*) $s[i] (1 \leq i \leq n)$. We use the terms *sequences* and *strings* interchangeably, and *letters*, *characters*, and *items* interchangeably. A *subsequence* is derived from a sequence by deleting some letters without changing the order of remaining letters. We use $|s|$ to denote the *length* of s (which is n here).

Given two sequences, such as $s_1 = AGCAT$ and $s_2 = GTACT$, the *Longest Common Subsequence* (LCS) problem is to find the longest subsequence common to both sequences. For example, $LCS(s_1, s_2) = GAT$. There can be more than one LCS (e.g., ACT is another one). The LCS algorithm is based on a recursive formulation as follows [4]:

$$l(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ l(i-1, j-1) + 1 & \text{if } s_1[i] = s_2[j] \\ \max(l(i, j-1), l(i-1, j)) & \text{if } s_1[i] \neq s_2[j] \end{cases} \quad (1)$$

where $l(i, j)$ denotes the length of LCS for $s_1[1 \dots i]$ and $s_2[1 \dots j]$. Line 2 of Equation (1) is for the case when the last characters of these two strings match, while line 3 is when they do not match. Clearly $l(|s_1|, |s_2|)$ is the final LCS length. Fig. 2(a) below shows the table of the dynamic programming algorithm resulted from this formulation, where the arrows are used to "trace back" the common subsequence. For instance, the highlighted cells are where the two sequences have match-

ing letters (line 2 of Equation (1)). The table can be filled in from the top-left corner to bottom-right. Value 3 in the bottom-right cell is the LCS length, and the subsequence can be retrieved by following the arrows.

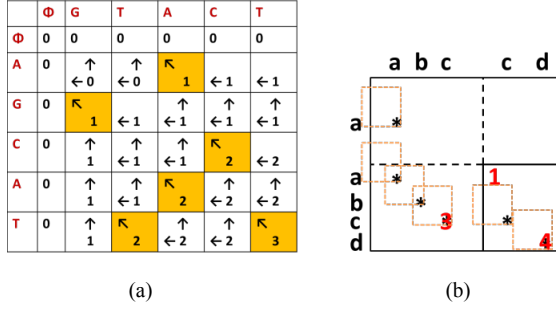


Fig. 2. (a) The classical algorithm to compute LCS. The highlighted cells are where two sequences have matching letters. (b) Illustrating the complicity of WLCS. Consider the solid rectangle window at bottom right. One cannot use LCS results $4 - 1 = 3$; the actual WLCS should be cd (of length 2).

B. Problem Formulation

For $1 \leq i \leq n$, each letter $s[i]$ in a sequence s has a non-decreasing timestamp $t[s[i]]$. A window w_j of size w ending at letter $s[b]$ consists of a set of consecutive letters $s[a], s[a+1], \dots, s[b]$, where the end timestamp $te(w_j) = t(s[b])$, the start timestamp $ts(w_j) = te(w_j) - w$, and $t[s[a-1]] < ts(w_j) \leq t[s[a]]$ (for boundary condition we have $t[s[0]] = -\infty$). We say that window w_1 precedes window w_2 (or w_2 follows w_1) if $te(w_1) < ts(w_2)$, and that windows w_1 and w_2 are non-overlapping if w_1 precedes w_2 or w_2 precedes w_1 . Although this is time-based window semantics, it is general enough to accommodate count-based windows by simply having timestamps as natural numbers.

Definition 1 (WCLCS). We have as input two event sequences s_1 and s_2 where $|s_1| = m$, $|s_2| = n$. The Window-Chained Longest Common Subsequence (WCLCS) problem is to find a chain of windows $w_{11}, \dots, w_{1c}$ in s_1 (where w_{1i} precedes $w_{1,i+1}$, and each has size w) and a chain of equal number of windows $w_{21}, \dots, w_{2c}$ in s_2 (where w_{2i} precedes $w_{2,i+1}$, and each has size w), such that $\sum_{i=1}^c |LCS(w_{1i}, w_{2i})|$ is maximized, subject to three constraints: (1) minimum density constraint: for each $1 \leq i \leq c$, $\frac{|LCS(w_{1i}, w_{2i})|}{w} \geq d_{min}$; (2) maximum gap constraint: $ts(w_{ji}) - te(w_{j,i-1}) \leq g_{max}$ ($j = 1, 2$ and $2 \leq i \leq c$); and (3) maximum shift constraint: $|ts(w_{1i}) - ts(w_{2i})| \leq s_{max}$ ($1 \leq i \leq c$).

The parameter values are based on application semantics. Not all constraints need to be present. $d_{min} = 0$, $g_{max} = \infty$, and $s_{max} = \infty$ correspond to the absence of the three constraints, respectively. WCLCS can be considered as a generalization of LCS, in that LCS is a special case where all three constraints are absent (and the window size can be arbitrary).

III. PER WINDOW LONGEST COMMON SUBSEQUENCE

WCLCS is a complex optimization problem. Our general strategy is to break it down into two sub-problems and solve each. At a high level, we get a set of candidate windows in the

first sub-problem, and compute the optimal chain from the candidate windows in the second sub-problem.

Definition 2 (Matching Point, Matching Window). For sequences s_1 and s_2 , consider two matching characters at positions i and j , respectively, i.e., $s_1[i] = s_2[j]$. We call such a pair (i, j) a matching point – as it corresponds to a point in a two-dimensional space. Each pair of windows w_{1i} and w_{2i} in Definition 1 thus corresponds to a two-dimensional window in this space, which is called a matching window.

In Fig. 2(a), highlighted cells are matching points. In Fig. 2(b), each $*$ is a matching point, and each dotted square is a matching window. As each matching window always has a last pair of matching letters, it suffices to consider the set of candidate matching windows W , where each $w_i \in W$ corresponds to a matching point p_i , in that p_i is w_i 's bottom-right corner (i.e., its last matching point). This is illustrated in Fig. 2(b) where each dotted matching window corresponds to a matching point at its bottom-right corner. These candidate matching windows in W may overlap with each other.

Thus, we decompose WCLCS into two sub-problems: (1) get per-window longest common subsequence (WLCS) for a window set W , and (2) select a subset of windows from W that satisfy Definition 1. We focus on sub-problem (1) with two approaches, and leave sub-problem (2) to Section IV.

A. The First Approach

We first propose a non-trivial extension of the LCS algorithm for WLCS. The difference between LCS and WLCS is that WLCS is only interested in matching counts in a limited horizon, namely within a window of size w in both sequences, whereas LCS seeks the maximum over the whole sequences.

However, this is not trivial. For example, the simple solution of subtracting the count in the top-left cell from that in the bottom-right cell of a two-dimensional window within the dynamic programming table of Fig. 2(a) does not work. This is illustrated in Fig. 2(b), where $s_1 = aabcd$, $s_2 = abccd$, and the window in question is the bottom-right solid rectangle (i.e., the last 4 letters of s_1 and last 2 letters of s_2). The red numbers show the counts in some cells if we follow the LCS algorithm in Fig. 2(a). Simply using $4 - 1 = 3$ to get WLCS length is incorrect, as the correct WLCS should be cd of length 2.

The key idea of our algorithm is as follows. Instead of a single count per cell in Fig. 2(a), we keep track of triples (t_1, t_2, c) , where c is the count, t_1 (t_2 , resp.) is the timestamp of the first matching letter in s_1 (s_2 , resp.) that contributes to the count c . Such a triple is copied between two adjacent cells based on the conditions in Equation (1). A triple is not copied further if either t_1 or t_2 is too old (beyond the window size w for a new cell). Then the maximum c among all triples in a cell is its WLCS length. PERWINLCS1 is presented below.

Line 3 is to see if it is a matching point. If so, it is similar to line 2 of Equation (1), and we iterate through each triple in the up-left neighbor cell (line 4), copy it over, and increment its count (line 7), unless this triple expires (lines 5-6). A triple is explained earlier; it is essentially a matching window whose first matching point is (i, j) . In line 8, we create a new triple with count 1, as (i, j) starts a new matching window.

Algorithm PERWINLCS1 (s_1, s_2)

Input: s_1, s_2 : two sequences of length m and n , respectively

Output: per window LCS between s_1 and s_2

```

1:  for each  $i \leftarrow 1$  to  $m$  do           //each letter in  $s_1$ 
2:    for each  $j \leftarrow 1$  to  $n$  do       //each letter in  $s_2$ 
3:      if  $s_1[i] = s_2[j]$  then           //a matching point
4:        for each  $(t_1, t_2, c) \in C[i-1, j-1]$  do
5:          //if the triple expires, then skip it
6:          if  $t(s_1[i]) > t_1 + w$  then continue
7:          if  $t(s_2[j]) > t_2 + w$  then continue
8:          //copy it over and increment count
9:           $C[i, j] = C[i, j] \cup (t_1, t_2, c + 1)$ 
10:         //create a new triple (matching window)
11:          $C[i, j] = C[i, j] \cup (t(s_1[i]), t(s_2[j]), 1)$ 
12:      else                               //( $i, j$ ) is not a matching point
13:        //copy/merge triples from two adjacent cells
14:         $C[i, j] = C[i, j-1] \cup C[i-1, j]$ 
15:      the max  $c$  value in  $C[i, j]$  is its WLCS length

```

On the other hand, if (i, j) is not a matching point (line 9), we copy and merge triples from two neighboring cells. There are two subtle details we omit here for clarity: **(1)** Like in lines 5-6, if a triple expires, we never need to copy it to (i, j) ; **(2)** A triple (t_1, t_2, c) can be removed from a cell if there is another triple (t'_1, t'_2, c') in the same cell such that $t_1 \leq t'_1$, $t_2 \leq t'_2$, and $c \leq c'$. This is easy to prove. (t'_1, t'_2, c') dominates (t_1, t_2, c) , as the former is no older (i.e., expires no sooner) and has a greater count (or the same).

Finally, in line 11, we can simply pick the maximum c among all triples in a matching-point cell to be its WLCS. Note that “tracing back” the common subsequence is easy by recording an “arrow” for each triple to indicate where it comes from. The worst case time complexity of PERWINLCS1 is $O(m \cdot n \cdot c_w^2)$, where c_w is the maximum number of letters in a window of a sequence, as each cell at most has $O(c_w^2)$ triples.

	a1	b2	c3	c4	d5
a1	(1,1,1)	(1,1,1)	(1,1,1)		
a2	(1,2,1)	(1,2,1)	(1,2,1)		
b3	(1,2,1)	(1,2,2) (2,3,1)	(1,2,2) (2,3,1)	(2,3,1)	
c4	(1,2,1)	(1,2,2) (2,3,1)	(1,2,3) (2,3,2) (3,4,1)	(2,3,2) (4,4,1)	(4,4,1)
d5		(2,3,1)	(2,3,2) (3,4,1)	(2,3,2) (4,4,1)	(4,4,2) (5,5,1)

(a)

	a1	c2	a3	b4	c5
b1				*	
a2	*		*		
c3		*			*
b4				*	
a5	*		*		

(b)

Fig. 3. (a) the PERWINLCS1 algorithm and (b) the PERWINLCS2 algorithm.

Example 5. Consider two sequences $s_1 = abccd$ and $s_2 = aabcd$ as shown horizontally (x -axis) and vertically (y -axis) in Fig. 3(a), resp., where the subscript of a letter denotes its timestamp. Let window size w be 3. The figure shows the triples. Let us take the first highlighted cell at $x = 3, y = 4$ (i.e., $C[3, 4]$) as an example. It is a matching point as c_3 in s_1 matches c_4 in s_2 . Lines 4-7 of the algorithm copy the two triples from cell $C[2, 3]$ and increment their counts, which gives the first two triples in $C[3, 4]$: $(1, 2, 3)$, $(2, 3, 2)$. Line 8 also adds a new triple $(3, 4, 1)$. The maximum count among these

three triples is 3 (underlined), which is the WLCS of the size 3 window ending at $x = 3, y = 4$. The second highlighted cell $C[3, 5]$ is an example of non-match-point. Line 10 merges the triples from $C[3, 4]$ and $C[2, 5]$. Among them, $(1, 2, 3)$ expires (for $w = 3$), while $(2, 3, 1)$ is dominated by $(2, 3, 2)$ and is removed. This results in the two triples in $C[3, 5]$, where the maximum count 2 is WLCS for the window ending at that cell.

B. Approach 2: Matching-point Based Algorithm

One observation from PERWINLCS1 is that the matching points between two sequences are important – the highlighted cells in Fig. 2(a) and the stars in Fig. 2(b). Only matching points have newly incremented counts in $C[i, j]$, while non-matching points have counts moved in from the two cells at previous row/column. This inspires us to devise an algorithm that efficiently obtains the matching points first, and then use a spatial index to efficiently retrieve all matching points in each matching window, from which we can calculate the per-window LCS. The algorithm, PERWINLCS2, is shown below.

Algorithm PERWINLCS2 (s_1, s_2)

Input: s_1, s_2 : two sequences of length m and n , respectively

Output: per window LCS between s_1 and s_2

```

1:  sort  $s_1$  and  $s_2$  respectively into (value,  $t$ ) pairs
2:  merge sorted lists, put matching points  $(t_1, t_2)$  in a spatial index  $\mathcal{P}$ 
3:  for each matching point  $(t_1, t_2) \in \mathcal{P}$  do
4:    rectangle box  $B \leftarrow [(t_1 - w, t_2 - w), (t_1, t_2)]$ 
5:     $M \leftarrow$  retrieve from  $\mathcal{P}$  all matching points in box  $B$ 
6:    sort points in  $M$  in increasing order of  $t_1, t_2$ 
7:    for each  $i \leftarrow 1$  to  $|M|$  do
8:       $c[i] \leftarrow 1$ 
9:      for each  $j \leftarrow 1$  to  $i - 1$  do
10:       if  $M[j].t_1 < M[i].t_1$  &  $M[j].t_2 < M[i].t_2$  &
11:          $c[j] + 1 > c[i]$  then
12:            $c[i] \leftarrow c[j] + 1$ 
13:     $c[|M|]$  is WLCS length at matching point  $(t_1, t_2)$ 

```

In line 1, we sort each of the two sequences in the order of the value of the letters, rather than timestamp (t). When either sequence is too large to fit in memory, an external sort algorithm is used. Then in line 2, we merge the two sorted lists and get matching points. Each matching point can be identified as (t_1, t_2) . Here t_1 (t_2 , resp.) is the timestamp of the letter c_1 (c_2 , resp.) from s_1 (s_2 , resp.), where c_1 and c_2 have the same value. Line 2 puts these matching points in a two-dimensional spatial index $\mathcal{P}$ (e.g., an R-tree [12]), where the two dimensions being indexed are t_1 and t_2 .

Then for each matching point (t_1, t_2) (line 3), we retrieve all the matching points within the window whose bottom-right corner is (t_1, t_2) , i.e., within the rectangle/window $[(t_1 - w, t_2 - w), (t_1, t_2)]$ in line 4, into M (line 5). By the definition of WLCS, the remaining task is to compute the LCS within this rectangle using M . From the nature of the recursive Equation (1), as shown in Fig. 2(a), we can associate a count $c(P)$ with each matching point P , and it holds that $c(P) = \max_{Q \in M, Q.t_1 < P.t_1, Q.t_2 < P.t_2} c(Q) + 1$. That is, there is an increment happening at P over the previous maximum count (among all matching points Q that have a smaller t_1 and a

smaller t_2). For instance, this LCS computation using matching points can be easily verified in Fig. 2(a) and (b). This is exactly what lines 6-12 do. Line 6 sorts the matching points in M in t_1 -major and t_2 -secondary order. For each point in this order (line 7), lines 8-12 carry out the iterative increment as described above. Finally, the last point in M must be (t_1, t_2) itself, and its count $c[M]$ is the WLCS of the window.

The time complexity of PERWINLCS2 is $O(m \log m + n \log n + c_M(\log c_M + c_m^2))$, where c_M and c_m are the total number of matching points and that in a window, respectively. The first two terms are for sorting the two sequences, and the remaining term is for going through the matching points, looking up the index, and computing the WLCS.

Example 6. Fig. 3(b) shows an example where $s_1 = acabc$ (horizontally, x -axis) and $s_2 = bacba$ (vertically, y -axis). The subscripts denote timestamps. All matching points are marked by stars. The loop at lines 3-12 of the algorithm iterates through each of these matching points and computes a WLCS value for the window that ends at that point. Let us take one of the matching points (5, 3) as an example. It is marked as the red star and the window that ends at it is the green highlighted rectangle in Fig. 3(b). M in line 5 has three points ($|M| = 3$), and line 6 sorts them in column-major order: (3, 2), (4, 1), (5, 3). The loop in lines 7-11 calculates $c[1] = 1$, $c[2] = 1$, and $c[3] = 2$. Line 12 in the end gets $c[3] = 2$ as the WLCS length for the highlighted window.

IV. WINDOW-CHAINED LONGEST COMMON SUBSEQUENCE

The *minimum density* constraint d_{min} and the *maximum shift* constraint s_{max} in Definition 1 (conditions 1 and 3) can be used as a filter amid the WLCS algorithms in Section III. For example, as a pre-filter of PERWINLCS2, a window can be skipped if the number of matching points in it (which is an upper bound of WLCS) is less than $d_{min}w$. A matching point (t_1, t_2) does not need to be considered if $|t_1 - t_2| > s_{max}$. It can be ensured that all remaining windows, a set W , satisfy d_{min} and s_{max} . The remaining task for WCLCS is as follows:

Remaining WCLCS Problem: Select a chain C of windows from W , subject to the maximum gap constraint (condition 2 of Definition 1), such that $\sum_{i=1}^{|C|} |\sigma_i|$ is maximized, where $c = |C|$ is the number of windows in C , and $|\sigma_i|$ is the WLCS length of the i 'th window in C (obtained in Section III).

A. WCLCS Algorithms

We convert the WCLCS selection into a graph problem. A *vertex-weighted* directed graph $\mathcal{G} = (V, E, W_v)$ is constructed as follows, illustrated in Fig. 4. Each window in W corresponds to a vertex $v \in V$, and its WLCS length corresponds to v 's weight in the weight function W_v . A directed edge $v_1 \rightarrow v_2$ is added if v_2 's window follows v_1 's, and they satisfy the gap constraint in Definition 1. Then the WCLCS problem is to find the longest path in $\mathcal{G}$, i.e., a directed path in $\mathcal{G}$ with the maximum total vertex weight.

In Fig. 4, each rectangle is a window in W , where the number in the bottom-right corner is its WLCS length. We also use this number to identify the window for succinctness and clarity. In the middle graph $\mathcal{G}$, each window maps to a vertex.

There is an edge from vertex 8 to 3 as window 3 follows window 8 and is within g_{max} . There is no edge from 6 to 3 since the two windows overlap (vertically). As another example, there is no edge from 8 to 5 since they are beyond the maximum gap allowed. It is easy to see that in this simple example, the longest path in $\mathcal{G}$ is $8 \rightarrow 9 \rightarrow 5$ with a total weight 22.

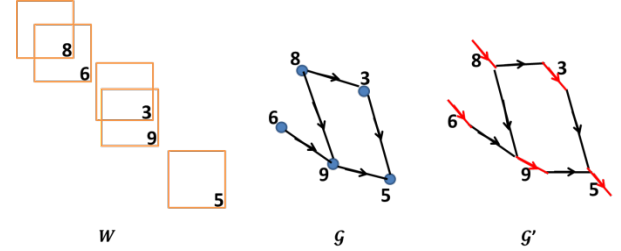


Fig. 4. Illustrating the process of converting the remaining WCLCS problem into a graph problem. We are given a set of windows W with their respective WLCS labeled in the window. Each window maps to a weighted vertex in $\mathcal{G}$. We further convert $\mathcal{G}$ into another graph $\mathcal{G}'$, where each weighted vertex is converted to a weighted red edge in $\mathcal{G}'$. The problem is to find the longest path.

To solve this graph problem, we further convert graph $\mathcal{G}$ into another graph $\mathcal{G}' = (V', E', W_e)$, which is an *edge-weighted* directed graph. The reason is that existing shortest path algorithms work on edge-weighted graphs. Each vertex $v \in V$ becomes two vertices v_{in} and v_{out} in V' , and we add a directed edge (v_{in}, v_{out}) whose *weight* is the vertex weight of v in $\mathcal{G}$. Moreover, an edge $(u, v) \in E$ becomes an edge (u_{out}, v_{in}) in E' , with weight 0. It is easy to prove (by contradiction) that the longest path in $\mathcal{G}$ (in terms of vertex weights) corresponds to a longest path in $\mathcal{G}'$ (in terms of edge weights), which is the goal of the WCLCS problem. This is again illustrated in Fig. 4, where a vertex in $\mathcal{G}$ maps to a red weighted edge in $\mathcal{G}'$, and the original black edges have weight 0.

Preliminary on single-source shortest path (SSSP). The SSSP problem is to find all the shortest paths from a particular vertex s in the graph to every other vertex. When negative edge weights are allowed, the Bellman-Ford algorithm is a commonly used one for SSSP. When it is a directed acyclic graph (DAG), there is an efficient $O(V + E)$ algorithm as follows. Let every vertex $v \in V$ have a label $L(v)$. Initially $L(s) = 0$ and $L(v) = \infty$ for any $v \neq s$. Define the *relaxation* operation on an edge $u \rightarrow v$ as: if $L(v) > L(u) + w(u \rightarrow v)$, then set $L(v) = L(u) + w(u \rightarrow v)$. The SSSP algorithm for DAG is simply to iterate over each vertex u in a topological sort order, and for each $v \in Adj[u]$ (i.e., outgoing neighbor), relax edge $u \rightarrow v$. In the end, $\forall v \in V$, $L(v)$ is the shortest distance from s to v . We refer the reader to [7] for details.

Let us intuitively understand how we arrive at an efficient algorithm for the longest path problem in $\mathcal{G}'$. The key ideas are:

- (1) Add an auxiliary vertex s (source), and an edge from s to every vertex in $\mathcal{G}'$ with weight 0. This converts WCLCS to a *single-source* longest path problem. With all weights negated, it then becomes an SSSP problem.
- (2) This is a DAG. Let us represent a window in W by the matching point (x, y) in its bottom-right corner. If there is an edge from (x_1, y_1) to (x_2, y_2) , it must be $x_1 < x_2$ and $y_1 < y_2$; hence no cycle is possible.

- (3) Iterating through the windows in y -major x -secondary order corresponds to a topological sort order of the DAG.
- (4) We do not need to explicitly construct $\mathcal{G}'$. Let each window $w_i \in W$ have a label $L(w_i)$. As we iterate over each window w_i in y -major order in (3), we just find which preceding window(s) w_j can reach w_i observing g_{max} , and “relax” the edge(s) as in the SSSP algorithm.

There are two cases: (1) $g_{max} = \infty$, i.e., there is no gap constraint, and (2) otherwise. We consider case (1) first. When we scan the windows in order, consider one of them, w_i , as shown in the small solid rectangle in Fig. 5(a). Without a gap constraint, any window in the top-left rectangle (with two edges dashed) in Fig. 5(a) can be a preceding neighbor of w_i , and we need to relax the edges between them. The edge being relaxed is really the “red” edge in Fig. 4 corresponding to w_i (as it has a non-zero weight). In this case, we only need to maintain a row of “largest label” we have seen so far to the left of each column (x value), as we scan in row-major (y -major) order, illustrated in the top vector in Fig. 5(a). This is because our “relaxation” only increases the label of a window (which is equivalent to decreasing the label for negated weights in the original SSSP algorithm).

Let $k = |W|$, the number of windows in W . Let k_x and k_y be the number of distinct x and y values, respectively, for all the bottom-right corner points of the k windows in W . We identify a window by the matching point (x, y) in its bottom-right corner. Let $L(x, y)$ denote a label for window (x, y) , and $w(x, y)$ denote the per-window LCS length (WLCS) of window (x, y) . The algorithm WCLCS-ANYGAP is shown below.

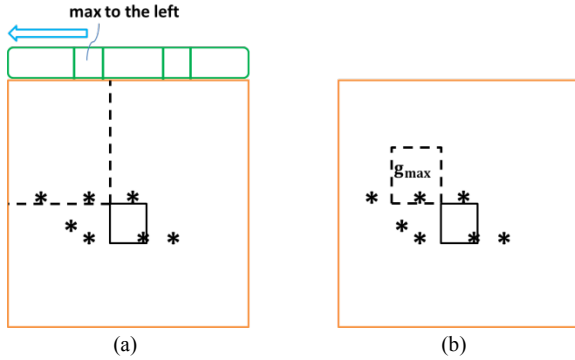


Fig. 5. Illustrating the WCLCS algorithm when (a) there is no gap constraint, and when (b) there is a maximum gap constraint g_{max} .

In line 1, we create the “largest label” vector l as discussed above, and initialize all entries to 0. Line 4 starts a loop that iterates through each window (x, y) (identified by its bottom-right matching point) in a y -major order (i.e., row by row). Lines 5-8 incrementally update vector l up to the 2-dimensional point $(x - w, y - w)$, i.e., right before the current window starts (as we cannot have overlaps between two windows in the final chain). Specifically, lines 6-7 update an entry in l if there is a window with a greater label, and line 8 simply propagate the same value to the right otherwise. Line 9 computes and updates the label for the current window. In the end, the window with the largest label is the final one in the required chain, and that label is the maximum length.

Algorithm WCLCS-ANYGAP (W)

Input: W : a set of windows with their WLCS values

Output: optimal WCLCS

```

1: create array  $l$  of size  $k_x$  //max label to the left
2:  $maxL \leftarrow 0$  //maximum label initialized to 0
3:  $(x_p, y_p) \leftarrow (0, 0)$  //previous window initialized to null
4: for each window  $(x, y) \in W$  in  $y$ -major order do
5:   for each window  $(x', y')$  between  $(x_p - w, y_p - w)$ 
      and  $(x - w, y - w)$  do
6:     if  $L(x', y') > l[x']$  then
7:        $l[x'] \leftarrow L(x', y')$ 
8:     else  $l[i] \leftarrow l[i - 1]$  for each  $i$  in between
9:    $L(x, y) \leftarrow l[x - w] + w(x, y)$ 
10:   $maxL \leftarrow \max(L(x, y), maxL)$ 
11:   $(x_p, y_p) \leftarrow (x, y)$ 
12: return  $maxL$ 

```

It is easy to see that the amortized cost of WCLCS-ANYGAP is $O(|W|)$, even though it has a double loop. This is because the inner loop (lines 5-8) also only goes through each window once throughout the algorithm.

Example 7. Fig. 6(a) shows an example where $s_1 = abccd$ (horizontally, x -axis) and $s_2 = aabcd$ (vertically, y -axis), and the subscripts are timestamps. Let $w = 2$. The vector l is on the top initialized to all 0's. Matching windows W are identified by matching points marked as stars in the figure. As the algorithm scans them in y -major (row major) order, we write the label of the window in the same cell. For the first three windows (stars), $(x - w, y - w)$ is still outside the index range of the vector l , and l stays all 0's; hence the labels of the first three windows are just their WLCS respectively (1, 1, 2). For the 4th window ($x = 3, y = 4$), we are able to update $l[1] = 1$, and effectively all following entries in l are 1 too (as it is non-decreasing). Thus, the 4th window gets a label $1 + 2 = 3$ (its WLCS is 2). Continue this until we reach the last window ($x = 5, y = 5$). Now the l vector can be updated up to the 3rd window and gets value 2 from $l[2]$ onwards. As a result, the last window gets a label $2 + 2 = 4$, which is the WCLCS length. The two red windows are the window chain.

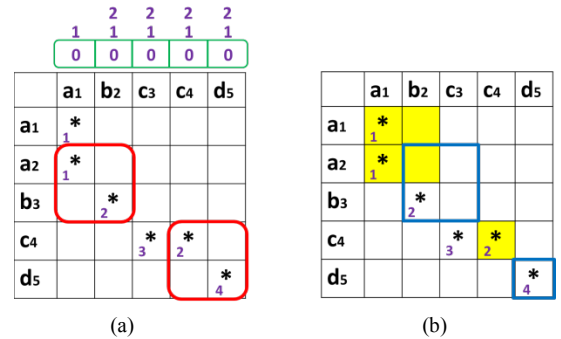


Fig. 6. Illustrating the execution of (a) WCLCS-ANYGAP and (b) WCLCS-MAXGAP on the same two sequences, where the subscripts are timestamps.

Now consider the case that there is a constraint g_{max} . Unlike the previous case, for a particular match window (x, y) , the location of its preceding window (such that there is a directed edge between them) is limited by g_{max} . More precisely, (the bottom-right corner of) a preceding window must be in the rectangle $[(x - w - g_{max}, y - w - g_{max}), (x - w, y - w)]$,

as $(x - w, y - w)$ is the start position of the current window. This is illustrated in Fig. 5(b). We use a spatial index to efficiently search for windows. The algorithm is shown below.

Algorithm WCLCS-MAXGAP (W)

Input: W : a set of windows with their WLCS values

Output: optimal WCLCS

```

1:  $maxL \leftarrow 0$  //maximum label initialized to 0
2: initialize a two-dimensional spatial index  $\mathcal{P}$ 
3: for each window  $(x, y) \in W$  in  $y$ -major order do
4:    $B \leftarrow [(x - w - g_{max}, y - w - g_{max}), (x - w, y - w)]$ 
5:    $P \leftarrow$  set of predecessor windows in box  $B$  from  $\mathcal{P}$ 
6:    $L(x, y) \leftarrow \max_{(x', y') \in P} L(x', y') + w(x, y)$ 
7:    $maxL \leftarrow \max(L(x, y), maxL)$ 
8:   store  $(x, y)$  in  $\mathcal{P}$ 
9: return  $maxL$ 

```

Line 2 initializes a two-dimensional index for storing and searching for preceding windows. As we iterate through each window (x, y) in y -major order, the rectangle B in line 4 is the search range discussed above. Lines 5-6 retrieve these preceding windows, and the one with the maximum label is used to calculate and update the label of the current window, which is stored in the index $\mathcal{P}$ in line 8. The window with the maximum label is the last window in the required chain.

Example 8. We use the same two sequences as in Example 7 and $w = 2$, as shown in Fig. 6(b), except that we now add the maximum gap constraint $g_{max} = 1$. As we go through the matching windows (identified by the matching points shows as stars), we look up a spatial index for preceding windows. For example, when the current window is $(x = 4, y = 4)$, shown as the yellow highlighted cell, the rectangle B to be searched (line 4) is the yellow highlighted 2-by-2 region, in which the maximum label is 1, and hence the label of the current window is set to $1 + 1 = 2$ (line 6). Similarly, the last window (ending at the blue-bordered cell) has a search rectangle B in the blue-bordered 2-by-2 region with maximum label 2. Hence the last window gets a label $2 + 2 = 4$. The same two windows as in Fig. 6(a), red rectangles, are in the chain.

B. Informed Search and Exploration

WCLCS-MAXGAP can be costly as it may involve a large number of spatial index operations when the number of matching windows is large. Observe that we may consider this as a search problem – searching for a window chain that has the maximum total length, among a large number of all possible chains. We present a novel informed search and exploration algorithm with the goal of cutting down the window chain search and spatial index lookups. Our algorithm belongs to the category of A*search [13]. The basic idea is as follows.

We store a set of window chains (most of which are partial) in a priority queue $\mathcal{Q}$ (implemented by a heap), where the priority of a window chain C can be written as $f(C) = g(C) + h(C)$. $g(C)$ is the total matching length of the windows in C . However, C may be incomplete and there can be more windows added to it. Hence, we use $h(C)$ to represent an *upper bound* of the total matching length of the windows that could potentially be added to chain C . As a result, $f(C)$ is an upper bound of any potential window chain derived from C .

Initially, each window chain in the priority queue $\mathcal{Q}$ is just a single matching window, and we have $|W|$ chains in $\mathcal{Q}$. For a single window chain C , $g(C)$ is just the WLCS of that window. What about $h(C)$? What can we put as an upper bound of the matching length of future windows to be added to C ? The answer is that we can resort to the LCS length. This is because LCS does not consider any constraints, and is an optimistic upper bound. Let us look at Fig. 5(a) for an illustration. Consider a chain C that contains a single window (x, y) , shown as the small solid rectangle in Fig. 5(a). Suppose we only try to extend the chain in the up-left direction, i.e., only windows preceding (x, y) may be added to C , chosen from the top-left big rectangle area in the figure (whose two edges are dashed). The LCS length of this rectangle is an upper bound, $h(C)$. If we only have LCS lengths up to each matching point, we can set $h(C) = LCS(x, y) - w(x, y)$, where $LCS(x, y)$ is the LCS length from the beginning of two sequences to (x, y) , and $w(x, y)$ is the WLCS of window (x, y) .

Each time a chain C with the maximum priority (i.e., $f(C)$) is removed from $\mathcal{Q}$. We then check if C can be extended by looking up the spatial index for its preceding windows subject to the gap constraint. For each preceding window, we prepend it to C and add the new chain to $\mathcal{Q}$. Clearly $h(C)$ will be updated based on the LCS at this preceding window. If there is no preceding window, we set $h(C)$ to 0 and add C to $\mathcal{Q}$. A chain C with $h(C) = 0$ is complete and cannot be extended. Therefore, if a chain C removed from $\mathcal{Q}$ has $h(C) = 0$, it must be the WCLCS. This is because it has the highest priority/length (by the definition of priority queue), and because all priority (f) values in $\mathcal{Q}$ are an upper bound. That is, no future complete chain could possibly have a greater priority (i.e., matching length) than C . The algorithm is below.

Algorithm WCLCS-ASTAR (W)

Input: W : a set of windows with their WLCS values

Output: optimal WCLCS

```

1: for each  $(x, y) \in W$  do
2:    $C \leftarrow \{(x, y)\}$ 
3:    $g(C) \leftarrow w(x, y)$  //per window LCS length
4:    $h(C) \leftarrow LCS(x, y) - w(x, y)$ 
5:    $f(C) \leftarrow g(C) + h(C)$ 
6:   put  $C$  into priority queue  $\mathcal{Q}$  with priority  $f(C)$ 
7: while  $\mathcal{Q}$  is not empty do
8:    $C \leftarrow$  pop maximum priority element from  $\mathcal{Q}$ 
9:   if  $h(C) = 0$  then return  $C$  //goal state
10:   $(x, y) \leftarrow$  last window in  $C$ 
11:  if  $(x, y)$  is marked “visited” then continue
12:   $pre \leftarrow$  preceding windows of  $(x, y)$  in  $W$ 
13:  mark  $(x, y)$  “visited”
14:  if  $pre = null$  then
15:     $h(C) \leftarrow 0$ 
16:    put  $C$  back to  $\mathcal{Q}$  with new  $h(C)$  and  $f(C)$ 
17:  else
18:    for each  $(x_p, y_p) \in pre$  do
19:       $C' \leftarrow clone(C)$  and append  $(x_p, y_p)$ 
20:       $g(C') \leftarrow g(C) + w(x_p, y_p)$ 
21:       $h(C') \leftarrow LCS(x_p, y_p) - w(x_p, y_p)$ 
22:       $f(C') \leftarrow g(C') + h(C')$ 
23:      put  $C'$  into  $\mathcal{Q}$  with priority  $f(C')$ 

```

Lines 1-6 put all the single-window chains into the priority queue Q as discussed above. Lines 7-23 are the main loop of the algorithm, in which we keep popping the maximum priority (i.e., f value) chain from Q . In line 9, if it is a complete chain, i.e., it cannot be extended, then we return the chain as the result. Otherwise lines 10-13 look up the spatial index (as before) and get preceding adjacent windows (subject to g_{max}). Lines 11 and 13 imply that a window needs only be “expanded” at most once throughout the algorithm. We shall analyze this in Theorem 1. If there are no preceding windows, lines 14-16 update $h(C)$ to 0 and put C back to Q . Otherwise for each preceding window, lines 18-23 append it to C , update its $g(\cdot)$, $h(\cdot)$, and $f(\cdot)$ values, and add the new chain to Q .

C	8	6	3	9	5	5,9	5,3	5,9,8	5,9,6	5,9,6	5,9,8
$g(C)$	8	6	3	9	5	14	8	22	20	20	22
$h(C)$	1	6	11	12	23	12	11	1	6	0	0
$f(C)$	9	12	14	21	28	26	19	23	26	20	22
pop					(1)	(2)		(4)	(3)		

Fig. 7. Illustrating WCLCS-ASTAR where the matching windows are shown in Fig. 4. The LCS values at the bottom-right cell of each window are: $LCS(8) = 9$, $LCS(6) = 12$, $LCS(3) = 14$, $LCS(9) = 21$ and $LCS(5) = 28$. The last row shows the order in which window chains are popped out from Q .

Example 9. Let us use the matching windows in Fig. 4. Recall that the numbers 8, 6, 3, 9, 5 are their WLCS lengths, and they are also the IDs of these windows for succinctness. Suppose the LCS values at the bottom-right cell of each window are: $LCS(8) = 9$, $LCS(6) = 12$, $LCS(3) = 14$, $LCS(9) = 21$ and $LCS(5) = 28$. Fig. 7 shows the table chains that enter into the priority queue Q in order, and the last row shows the order in which four of the chains are popped out from Q (the last one 5, 9, 8 is the WCLCS result). Initially the algorithm puts the single-window chains (first five columns in Fig. 7) into Q , where $g(\cdot)$, $h(\cdot)$, and $f(\cdot)$ values are computed as in lines 3-5, and the $f(\cdot)$ values are their priorities. The highest priority is 28 (window 5); hence line 8 pops it out from Q . Window 5 has preceding adjacent windows 9 and 3; hence chains 5, 9 and 5, 3 enter Q (lines 18-23). Now the highest priority is 26, and chain 5, 9 is popped out. Then the next two columns in Fig. 7, chains 5, 9, 8 and 5, 9, 6 enter Q . Chain 5,9,6 is popped out, and window 6 has no preceding windows; hence its $h(\cdot)$ is changed to 0 and is put back to Q . The same thing happens to the next popped out chain 5, 9, 8. But after its $h(\cdot)$ is changed to 0, it still has the highest priority. Line 9 returns this result.

Since the priority queue can be large, we use the simple approach in SMA* [19] to temporarily remove “less promising” nodes from the search tree when it is over memory limit. We now show the correctness of WCLCS-ASTAR.

Theorem 1. The WCLCS-ASTAR algorithm gives the correct result. In particular, a matching window in the input W needs to be “visited” (i.e., spatial index lookup and expansion to multiple chains) at most once. The finally returned window chain is WCLCS.

Proof. First, as discussed earlier, throughout the algorithm, for a window chain C , $f(C) = g(C) + h(C)$ is an upper bound of the total matching length of any potential window chain C' that could be derived from C . Secondly, a window $w_i = (x, y)$

needs only be visited and expanded at most once (lines 11-13), regardless of how many chains share/cross w_i . Suppose that two chains C_1 and C_2 intersect at window w_i , and that C_1 reaches w_i first. Let the list of windows in C_1 and C_2 before w_i be L_1 and L_2 , respectively. It must be true that $g(L_1) \geq g(L_2)$, where the $g(\cdot)$ function is the sum of WLCS lengths of the windows in the list. This is because the $h(\cdot)$ function value is identical at w_i , and the fact that C_1 is popped from the priority queue first implies that $g(L_1) \geq g(L_2)$. Therefore, C_1 must be superior to (or at least the same as) C_2 up to w_i . Whatever choices may be after w_i , we can simply continue with C_1 and abort C_2 . In the end, the popped maximum priority chain C has $h(C) = 0$ and cannot be extended. As the priorities of all other chains in Q are upper bounds of their matching lengths, any future chains that they derive cannot be better than C . $\square$

The WCLCS-ASTAR algorithm is based on the “best-first” search principle and always tries to examine and expand the most promising window chain so far. Theorem 1 indicates that any matching window is looked up in the spatial index at most once. In fact, some window may never get visited and expanded before the optimal chain is returned. The saving stems from such pruning of the search space.

V. APPROXIMATION ALGORITHM

To further improve performance, we devise an approximation algorithm. The basic idea is that we sample the matching points with a certain ratio ρ . We then find the optimal window chain through the sample of matching points. Fewer matching points entail more efficient WCLCS algorithms. This is especially the case when the whole spatial index structure used by the algorithms fits in memory. The algorithm, WCLCS-APPROX, is presented below.

Algorithm WCLCS-APPROX (s_1, s_2)

Input: s_1, s_2 : two sequences of length m and n , respectively

Output: WCLCS

- 1: run PERWINLCS2, but a matching point is put in the spatial index $\mathcal{P}$ with probability ρ
- 2: run any of previous WCLCS algorithms, and let result window chain be C
- 3: **for each** window w_i in chain C **do**
- 4: find LCS of w_i within original s_1, s_2 using all the matching points in w_i
- 5: **return** C

Line 1 chooses to run PERWINLCS2 instead of PERWINLCS1 because sampling of matching points will speed up PERWINLCS2 itself but not PERWINLCS1. This is because all operations after line 1 of PERWINLCS2 will be faster due to fewer matching points and a smaller spatial index, while PERWINLCS1 still needs to go through all cells to propagate the triples. Our experiments in Sec. VI also show that PERWINLCS2 is significantly faster in general. The density constraint is adjusted to $\rho \cdot d_{min}$. In line 2, we can run any of the previously presented WCLCS algorithms over the resulting set of matching windows from line 1, and get a window chain.

Lines 3-4 iterate through each window in the chain and recover the original matching subsequences in it before the sampling of matching points. Since each 2-dimensional win-

dow in C corresponds to a window in s_1 and s_2 respectively, this step is easy to do and is efficient. We now show the accuracy guarantee.

Theorem 2. *Let the total number of matching characters in the window chain returned by WCLCS-APPROX be l , and that in the true WCLCS be l^* . Then for $0 < \delta < 1$, $\Pr[l < (1 - \delta)l^*] \leq e^{-\rho l \delta^2 / [(1-\delta)(5-3\delta)]}$.*

Proof. Define a random variable X as the length of the WCLCS found through the sample after line 2 of WCLCS-APPROX. Recall that l is the WCLCS length returned by WCLCS-APPROX. From Chernoff bound, for $0 < \delta_2 \leq 1$,

$$\Pr[X \geq (1 + \delta_2)\rho l] \leq e^{-\rho l \delta_2^2 / 3} \quad (2)$$

Next, define a random variable Y as the length of matching letters in the sample of the *true* WCLCS window chain. Again from Chernoff bound, for $0 < \delta_1 < 1$,

$$\Pr[Y \leq (1 - \delta_1)\rho l^*] \leq e^{-\rho l^* \delta_1^2 / 2} \quad (3)$$

We also have that $\Pr[l \leq \frac{1-\delta_1}{1+\delta_2} l^*] \leq \Pr[X \geq (1 + \delta_2)\rho l \cap X \leq (1 - \delta_1)\rho l^*]$ since the event on the RHS implies the event on the LHS. As $X \geq Y$ (by the definition of X , which is returned by the algorithm), we have $\Pr[l \leq \frac{1-\delta_1}{1+\delta_2} l^*] \leq \Pr[X \geq (1 + \delta_2)\rho l \cap Y \leq (1 - \delta_1)\rho l^*] \leq e^{-\rho l \delta_2^2 / 3 - \rho l^* \delta_1^2 / 2}$. Letting $\frac{1-\delta_1}{1+\delta_2} = 1 - \delta$, we then get $\Pr[l \leq (1 - \delta)l^*] \leq e^{-\rho l z}$, where $z = \frac{5\delta_1^2 + 2\delta^2 - 3\delta_1^2\delta - 4\delta\delta_1}{6(1-\delta)^2}$. z is maximized when $\frac{\partial z}{\partial \delta_1} = 0$, giving us $\delta_1 = \frac{2\delta}{5-3\delta}$. This gives the result in the theorem. $\square$

Theorem 2 indicates that a greater total matching length (l) gives a tighter accuracy bound. This is desirable since a great matching length is exactly when the approximation algorithm is most needed (e.g., recall that WCLCS-ASTAR needs to search a larger space when the window chain is long). Our experimental results in Section VI are especially encouraging: WCLCS-APPROX improves performance by over an order of magnitude while the accuracy is nearly perfect. Hence, this method is very effective in practice.

VI. EMPIRICAL STUDY

We perform a systematic empirical evaluation to answer the following questions:

- How much does the result of WCLCS differ from that of LCS (the classical longest common subsequence) in finding common events in real-world sequence data?
- We have proposed two algorithms for per-window longest common subsequence. How do they compare to each other and compare to the LCS algorithm?
- How do the overall WCLCS algorithms (without gap constraints and with gap constraints) perform under different parameters with real-world and synthetic data?
- How does the informed search and exploration algorithm help on performance?
- How much does the approximation algorithm improve the performance and what about its result accuracy?

A. Datasets and Setup

We use two real-world datasets and synthetic data to carry out our evaluation:

Stock data. We use the stock data of Western Digital (WDC) and IVE (S&P 500 Value Index) since September 2009, retrieved from [15]. The size of the WDC stock sequence is 1.1 GB, and the size of the IVE sequence is 100 MB. We limit the memory usage of our program to be no more than 400 MB, so that the data cannot fit in memory entirely.

Microsoft Research GPS data. This GPS trajectory dataset was collected in Microsoft Research Asia's Geolife project over five years [23]. A GPS trajectory is a sequence of time-stamped points, each of which contains the latitude, longitude and altitude (we only use latitude and longitude). This dataset contains 17,621 trajectories with a total distance of 1,292,951 kilometers and a total duration of 50,176 hours. These trajectories were recorded by different GPS loggers and GPS-phones. This dataset is 1.6 GB.

Synthetic data. We also generate some synthetic data based on the stock data in order to vary parameters such as the alphabet size of a sequence. We will describe how it is generated when we discuss the experiments below.

We have implemented all the algorithms presented in the paper in Java, as well as the Longest Common Subsequence algorithm for comparison in results and for a reference point in performance. We use JSI [14] for R-tree when the index can fit in memory, and an implementation of R*-tree [8] when it cannot, since these two implementations are the fastest spatial indices (among several that we can find) for in-memory and external memory, respectively. Moreover, we modify the open source code for external sorting [10] to meet the sorting needs in our algorithms. All experiments are performed on a machine with an Intel Core i7 2.50 GHz processor and an 8GB memory. As mentioned earlier, we limit the memory usage of our program to be no more than 400 MB.

B. Experimental Results

For the stock data, we first preprocess the WDC stock tick sequence and the IVE stock tick sequence, respectively, so that each data point signifies its *change direction* (increase/decrease) and *magnitude* (slight/significant) over the corresponding data point 24 hours ago (or one that is closest to that time). Thus, the alphabet size of the two sequences (WDC and IVE) is 4, with four letters 0, 1, 2, and 3 indicating *slight increase* (increase amount less than 0.5), *significant increase* (amount 0.5 or more), *slight decrease* (amount less than 0.5), and *significant decrease* (at least 0.5), respectively. The resulting sequences (WDC and IVE) indicate the “change trend” of WDC and IVE, rather than their absolute values.

We run WCLCS (with no gap constraint between two windows) over the two sequences to find out when the company Western Digital's stock (WDC) generally follows the same trend as the market it is in, S&P 500 (IVE). The mismatched part of the sequence further indicates when WDC's stock trend does not agree with IVE. Such results are useful for business analysis. Interestingly, we find from our results that the mismatched part typically follows significant changes or news of Western Digital, e.g., in March 2011 when the company agreed to acquire the storage unit of Hitachi, and in July

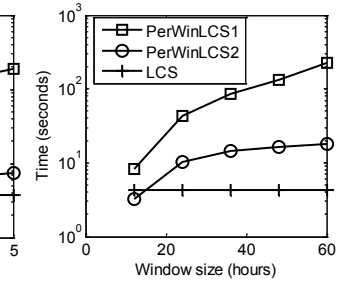
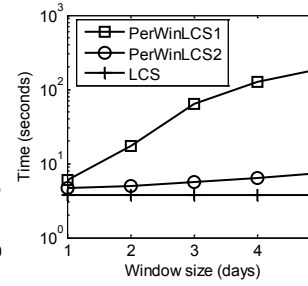
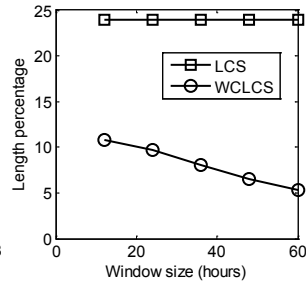
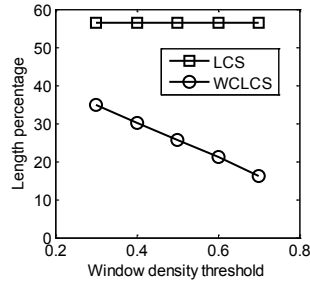


Fig 8 Match length comparison (Stock) Fig 9 Match length comparison (GPS) Fig 10 Per window LCS (Stock) Fig 11 Per window LCS (GPS)

2012 when the company announced the new “Red” series of drives.

By contrast, when we run LCS on the two sequences, we cannot make sense of the result, as the matching characters *scatter* and *touch almost everywhere* in the sequences. Any short piece of one sequence could always potentially find matches in the other sequence with arbitrary gaps in between. Figure 8 shows this point where we can see that the matching length of LCS is about 60% of the entire length of the shorter sequence (IVE), while total length of WCLCS is between 35% and 16% of the entire length as the minimum density threshold for each window varies between 0.3 and 0.7. The matching density here is based on the fraction of matches over all letters in a window. Thus, a higher threshold filters out more windows. The window size here is one day.

Similarly, we compare the result of WCLCS and LCS using the GPS dataset. We find that the vast majority (over 80%) of the locations in the dataset are within the 5th Ring Road – a large circle/rectangle area in Beijing. The latitude range of this area is [39.794820, 40.010787] and the longitude range is [116.238785, 116.546402]. We divide this rectangle into a 16x16 grid, where each of the 256 areas is about 0.9 mile by 1.1 mile, and is assigned a letter. Hence, we get a sequence by concatenating multiple trajectories together.

Analogous to Example 2, finding the WCLCS of two such sequences can identify two people’s close location sequences when they are likely participating in the same set of activities. For instance, we find that most of the matching location sequences are in Zhongguancun and Haidian district, as the matching sequences should be from two college students or Microsoft Research Asia employees there. On the other hand, the LCS result does not take into consideration the semantics of temporal window and match density, and may result in drastically different matching subsequences. Figure 9 shows the difference between the two in total matching length when the window size w of WCLCS varies from 12 hours to 60 hours, while the window density threshold is fixed at 0.4 (which is the default value in our experiments). The WCLCS length is greater when w is smaller because it is easier to satisfy the window density constraint with a smaller window, contributing to the total WCLCS length.

Recall that we devise two algorithms, PERWINLCS1 and PERWINLCS2, for per-window longest common subsequence. In this experiment, we verify that they produce the same results, and compare their performance. We first use the stock dataset, and vary the window size from 1 day to 5 days. Figure

10 shows the result in a semi-log plot. We can see that PERWINLCS2 is much faster than PERWINLCS1, especially when window size is large (over an order of magnitude faster). This is because when window size is large, the number of triples (t_1, t_2, c) in each cell $C[i, j]$ of PERWINLCS1 increases precipitously, while PERWINLCS2 uses look-ups from a spatial index and provides a much more scalable solution. We also compare their performance with that of the classical LCS as a reference point. LCS is faster as it does not need to calculate the maximum length per window.

We repeat the same experiment with the GPS dataset and show the result in Figure 11. Again we see that PERWINLCS2 can be an order of magnitude faster than PERWINLCS1, and LCS is in general the fastest. PERWINLCS2 can be slightly faster than (or about the same as) LCS when the window size is small and hence the number of match points within each window is small. In this case, the overheads of the two are about the same. In summary, this set of experiments show that PERWINLCS2 should be the preferred method to use than PERWINLCS1 for the first step of WCLCS, per-window LCS. We use PERWINLCS2 in the subsequent experiments.

We now evaluate the two versions of WCLCS algorithms, WCLCS-ANYGAP and WCLCS-MAXGAP, where the former is when there is no gap constraint, and the latter is when there is a maximum gap constraint. We first use the stock dataset and vary the maximum gap (between two windows) parameter g_{max} , the result of which is shown in Figure 12. For this dataset, our default window size is 2 days. The figure shows that WCLCS-ANYGAP is a few times faster than WCLCS-MAXGAP, and the execution time of WCLCS-MAXGAP increases slightly with g_{max} . This is because WCLCS-MAXGAP needs to look up and update an index for each window in the input, while WCLCS-ANYGAP only needs to sequentially scan each window in the input. Figure 13 shows a similar trend for the GPS dataset. In both figures, LCS is faster than the two versions of WCLCS algorithms. Moreover, the WCLCS algorithms for the GPS dataset are about twice slower than for the stock dataset due to more matching windows to be processed. In the following experiments, we vary other parameters.

We next examine the impact of alphabet size $|\Sigma|$ on performance. For this we use a synthetic dataset generated from the stock dataset. Recall that our experiments with the stock dataset use two sequences (IVE and WDC) with letters of alphabet size 4 for *slight/significant increase/decrease* compared to 24-hours ago, respectively. We now generate a syn-

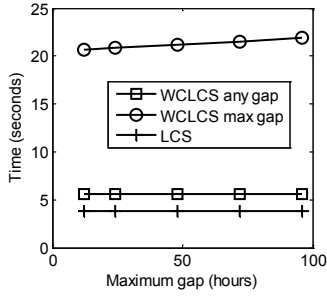


Fig 12 WCLCS two versions (Stock)

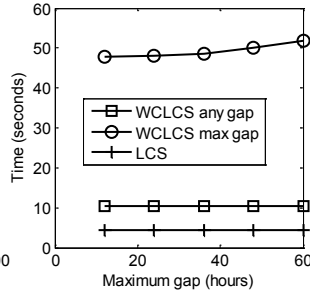


Fig 13 WCLCS two versions (GPS)

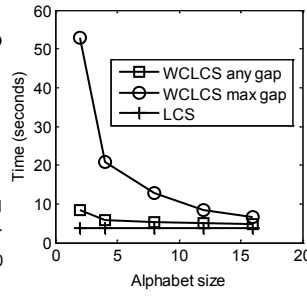


Fig 14 Varying $|\Sigma|$ (synthetic)

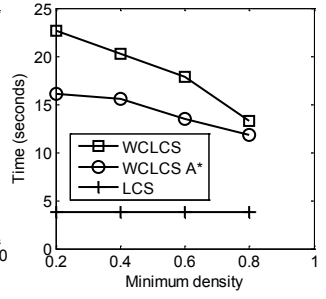


Fig 15 WCLCS A* (Stock)

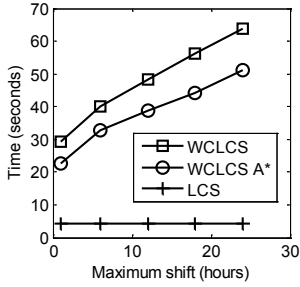


Fig 16 WCLCS A* (GPS)

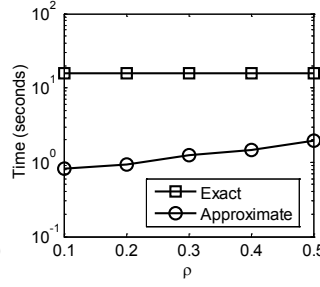


Fig 17 Approximation algorithm (Stock)

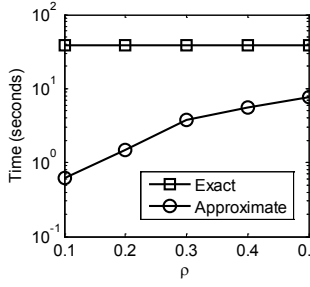


Fig 18 Approximation algorithm (GPS)

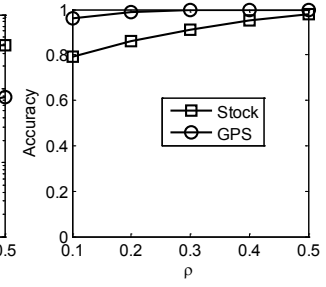


Fig 19 Accuracy vs. ρ

thetic dataset from this, but have different numbers of discretization levels ranging from 2 to 16. For example, when $|\Sigma| = 2$, we only have two levels: *increase* and *decrease*; when $|\Sigma| = 16$, we have 8 fine-grained levels for increase and 8 for decrease. The performance result for these various alphabet sizes is shown in Figure 14. When $|\Sigma|$ increases, the performance of WCLCS-MAXGAP improves drastically. This is because the chance of letter matching between two sequences precipitously decreases; so does the number of “matching points” in the algorithm. This has a significant effect on performance: the algorithm needs to iterate through fewer matching points; the spatial index is smaller and may even fit in memory; the number of matching points to be sorted is smaller. This experiment also informs us that efforts to reduce the number of matching points in the algorithm may considerably improve the performance.

In the next set of experiments, we examine the WCLCS-ASTAR algorithm, which is based on informed search and exploration. We first use stock data, and vary the minimum density threshold for each window between 0.2 and 0.8. This is shown in Figure 15. We can see that, since WCLCS-ASTAR cuts down on the number of match windows to be examined (recall that it tries to finish early), it improves the performance of WCLCS-MAXGAP. When the density threshold is really high, fewer match windows exist, which reduces the effect of WCLCS-ASTAR. We then perform a similar experiment using the GPS data, but vary the maximum shift parameter between two matching windows in the range from one hour to twenty-four hours. The result is shown in Figure 16. Again we see that WCLCS-ASTAR improves the performance due to the pruning in the search space. In addition, when the allowed maximum shift increases, both algorithms take longer time due to more matching points.

The last set of experiments has to do with the approximation algorithm that we devise. We first use the stock dataset, and apply the approximation algorithm over the best WCLCS algorithm so far, WCLCS-ASTAR. The performance result is shown in Figure 17, where the sampling rate ρ varies from 0.1 to 0.5 (the result accuracy will be shown later). We can see that the approximation algorithm here drastically improves the performance by an order of magnitude. The improvement is more when ρ is smaller. This is due to the significant reduction in number of matching points, which also results in a smaller spatial index used by the algorithm. We carry out the same experiment with the GPS dataset, and the result is shown in Figure 18, where a similar trend is seen.

While the approximation algorithm greatly helps performance, we would also like to check how much accuracy is lost to trade for the performance gain. We compute the ratio between the length of the WCLCS found by the approximation algorithm and that of the exact algorithm. The results for both datasets and various ρ values are shown in Figure 19. It shows that we can get very good accuracy (i.e., the total length of WCLCS found is almost identical to the true WCLCS) with the approximation algorithm while enjoying an order of magnitude improvement in speed. For instance, in the stock dataset, when we use $\rho = 0.5$, the accuracy is 0.98 while the speedup is 8 times. The accuracy is even more with the GPS dataset – when $\rho = 0.3$, the accuracy is still 100% with a speedup of over 10 times. The GPS dataset has an even better result than the stock dataset because it has more matching points and a longer WCLCS length, which makes sampling more effective and accurate, consistent with our analysis in Theorem 1. As a matter of fact, the case of a large number of matching points is precisely when we need the approximation algorithm the most, since the exact algorithm would be slow.

This set of experiments show that the approximation algorithm is very effective and is much needed for scaling up WCLCS.

Summary of results. Our experimental results in this section show that the WCLCS semantics produces quite different and more meaningful and useful common event matching results than LCS. As the first step of the WCLCS algorithm, PERWINLCS2 should be the preferred method to use rather than PERWINLCS1, as the former is much faster. WCLCS-MAXGAP is slower than WCLCS-ANYGAP, but it can be improved by using the informed search algorithm WCLCS-ASTAR. Finally, the approximation algorithm is very effective – it achieves very high accuracy and often over an order of magnitude speedup, especially for large sequences where the number of matching letters is high.

VII. RELATED WORK

Studying events in sequences is a very relevant problem in the big data era due to emergence of large amounts of sequence data, including system logs, business logs, and time series. There has been extensive study of event processing in sequences in both academia (e.g., [1, 22]) and industry (e.g., [6, 18]). There is also previous work on mining complex event patterns, such as [21, 17]. In [21], Seipel et al. study how to use temporal data mining in a declarative framework for analyzing log files for computer networks. In particular, they show how to derive temporal association rules from network management protocol messages. In [17], Mannila et al. study how to discover frequent episodes in sequences. However, to the best of our knowledge, no previous work studies the problem of finding common event subsequences from two long event sequences as we do.

There has been recent interest in sequence and sorting algorithms on modern processors, such as the one by Chandramouli and Goldstein [5]. Some of our algorithms use external sorting, and can potentially benefit from this work, although our goal is to solve a different problem. Querying and mining time series data has been an active research topic for a long time. In particular, representation of time series and their distance measures have drawn much attention (e.g., [9]). However, this line of work is fundamentally different from ours in that we are interested in extracting a meaningful long common event sequence from two sequences, but not in how different two time series are.

Study of longest common subsequence (LCS) has a long history, e.g., [16]. We refer the reader to [4] for an excellent survey of this topic. We have discussed at length why LCS does not meet the needs of our problem, and have compared our WCLCS semantics, results, and performance with LCS. Some variations of LCS work exist. For example, Ann et al. [3] study the problem of finding LCS that is also a supersequence of another input sequence P . Albert et al. study the problem of longest increasing subsequence (LIS) in sliding windows [2]. This has windows in semantics, but it is a completely different problem – there is only one sequence, and the

goal is to find the LIS. We are addressing a new problem on common event matching between two sequences, and hence our semantics differs significantly from all previous work.

VIII. CONCLUSION

In this paper, we study the problem of finding common events from two long sequences, which finds many applications in diverse domains. We propose a new semantics called WCLCS, and devise efficient algorithms by reducing it to a graph problem. Moreover, we devise novel performance improvement methods: one is based on informed search and exploration, while the other is an approximation algorithm. Our experimental results are enlightening and verify our approaches and algorithms.

ACKNOWLEDGMENT

This work was supported by the NSF, under the grants IIS-1149417, IIS-1239176, and IIS-1319600.

REFERENCES

- [1] J. Agrawal, Y. Diao, D. Gyllstrom, N. Immerman. Efficient Pattern Matching over Event Streams. In *SIGMOD*, 2008.
- [2] M. Albert, et al. Longest increasing subsequences in sliding windows. In *Theoretical Computer Science*, 321, 405-414, 2004.
- [3] H. Ann, C. Yang, C. Tseng, C. Hor. Fast algorithms for computing the constrained LCS of run-length encoded strings. In *TCS*, 2012.
- [4] L. Bergroth, H. Hakonen, T. Raita. A survey of longest common subsequence algorithms. In *IEEE SPIRE*, 2000.
- [5] B. Chandramouli and J. Goldstein. Patience is a Virtue: Revisiting Merge and Sort on Modern Processors. In *SIGMOD*, 2014.
- [6] B. Chandramouli, J. Goldstein, D. Maier. High-Performance Dynamic Pattern Matching over Disordered Streams. In *VLDB*, 2010.
- [7] T. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to Algorithms* (2nd ed.). MIT Press and McGraw-Hill, 2001.
- [8] Dimitris Papadias group. Java implementation of R*-tree. Downloaded from <http://www.chorochronos.org/?q=node/43>.
- [9] H. Ding, G. Trajcevski, P. Scheuermann, X. Wang, E. Keogh. Querying and Mining of Time Series Data: Experimental Comparison of Representations and Distance Measures. In *VLDB*, 2008.
- [10] External sorting. <https://code.google.com/p/externalsortinginjava/>.
- [11] C. Fleisher, B. Bensoussan. *Strategic & Competitive Analysis: Methods & Techniques for Analyzing Business Competition*. Prentice Hall, 2003.
- [12] A. Guttman. R-Trees: A Dynamic Index Structure for Spatial Searching. In *SIGMOD*, 1984.
- [13] P. Hart, N. Nilsson, B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. In *IEEE TSSC*, 1968.
- [14] JSI, Java Spatial Index. <http://jsi.sourceforge.net/>.
- [15] Kibot. <http://www.kibot.com/>.
- [16] D. Maier. The Complexity of Some Problems on Subsequences and Supersequences. In *JACM*, Vol. 25, No. 2, pp. 322-336, 1978.
- [17] H. Mannila, H. Toivonen, A. Verkamo. Discovering frequent episodes in sequences. In *KDD*, 1995.
- [18] Oracle Complex Event Processing: Lightweight Modular Application Event Stream Processing in the Real World. *A White Paper*, 2009.
- [19] S. Russell. Efficient memory-bounded search methods. In *Proceedings of the 10th European Conference on Artificial intelligence*, 1992.
- [20] A. Sachdev. Your Car Knows Everything. *Chicago Tribune*, Sep, 2011.
- [21] D. Seipel, P. Neubeck, S. Köhler, M. Atzmueller. Mining Complex Event Patterns in Computer Networks. *LNCS*, V. 7765, pp 33-48, 2013.
- [22] L. Woods, J. Teubner, G. Alonso. Complex Event Detection at Wire Speed with FPGAs. In *VLDB*, 2010.
- [23] Y. Zheng et al. Mining interesting locations and travel sequences from GPS trajectories. In *WWW*, 2009. <http://research.microsoft.com/en-us/downloads/b16d359d-d164-469e-9fd4-daa38f2b2e13/>.
- [24] <http://archive.ics.uci.edu/ml/datasets/Diabetes>.